

Sci-Tech Discovery Center — Interactive Signage

An interactive kiosk for the **Human Machine** exhibit at the Sci-Tech Discovery Center in Frisco, TX. A child scans a body-part artifact on the scanner, picks a language, and watches a short educational video. Then the screen automatically resets for the next visitor.

Supported languages: English · Spanish (Español) · Telugu

Table of Contents

For Staff & Operators (non-technical)

- [How the Kiosk Works](#)
- [What You Need](#)
- [One-Time Setup Guide](#)
- [Starting and Stopping the Kiosk](#)
- [Make the Kiosk Start Itself on Power-Up](#)
- [Swapping or Adding Videos](#)
- [NFC Tag Manager — Full Guide](#)

For Developers

- [Project Structure](#)
 - [Tech Stack](#)
 - [Architecture & API](#)
 - [Running in Development](#)
-

For Staff & Operators

How the Kiosk Works

1. The screen shows a prompt telling the visitor to scan their artifact.
2. The visitor holds an organ artifact up to the scanner. The screen changes to show three language choices.
3. The visitor (or a staff member) presses one of the three physical buttons — or taps a button on the screen — to pick English, Spanish, or Telugu. If nobody picks within 15 seconds, it automatically plays in English.
4. The video plays full-screen.
5. When the video ends, the screen goes back to step 1, ready for the next visitor.

What You Need

Hardware

Item	What it is
Raspberry Pi 5	The small computer that runs everything. Needs its power supply and a microSD memory card (32 GB or larger).
PN532 NFC Reader	The tag scanner.
3 push buttons	One for each language. Connected to the Pi with short wires.
Monitor or TV	Any screen with an HDMI port.
HDMI cable	To connect the Pi to the screen.
Jumper wires	Short wires used to connect the buttons to the Pi.

What gets installed automatically

You don't need to install anything manually. When you run the setup in Step 4 below, the Pi downloads and installs everything it needs on its own.

One-Time Setup Guide

These steps only need to be done once. After setup is complete, daily operation is just plugging the Pi in (if auto-start is enabled) or running one command.

Step 1 — Put the Operating System on the Memory Card

The Raspberry Pi needs an operating system installed on its memory card before it can do anything. Think of it like installing Windows on a new PC, except it's free and takes about 5 minutes.

1. On a regular computer, download and install the **Raspberry Pi Imager** from raspberrypi.com/software.
2. Insert the microSD card into your computer (you may need a USB card reader adapter).
3. Open Raspberry Pi Imager, choose **Raspberry Pi OS (64-bit)**, select your memory card, and click **Write**. This takes a few minutes.
4. Put the memory card into the Raspberry Pi, connect it to the monitor with HDMI, plug in a keyboard and mouse, and plug in the power cable.
5. The Pi will start up and walk you through a short setup wizard — connect to Wi-Fi and finish the steps it shows you.

Step 2 — Enable a Required Setting for the Scanner

The tag scanner needs a specific setting turned on before it will work. Here's how:

1. Open the Terminal app.
2. Type exactly the following and press **Enter**:

```
sudo raspi-config
```

If it asks for your password, type it and press Enter. The characters won't show up as you type.

3. Using the **arrow keys**:
 - Highlight **Interface Options** and press **Enter**
 - Highlight **SPI** and press **Enter**
 - Highlight **Yes** and press **Enter**
 - Highlight **Finish** and press **Enter**
4. When it asks if you want to reboot, choose **Yes**. The Pi will restart.
5. Once it's back on the desktop, continue to Step 3.

Step 3 — Download the Project Files

Now you'll download all the kiosk software onto the Pi.

1. Open the Terminal
2. Type the following line and press **Enter**. This changes the working directory to the desktop.

```
cd ~/Desktop
```

3. Type the next line and press **Enter**. This downloads the kiosk software:

```
git clone https://github.com/UTDallasEPICS/Sci-Tech-Discovery-Center-Interactive-Signage.git
```

4. Type this and press **Enter**. This opens the folder you just downloaded:

```
cd Sci-Tech-Discovery-Center-Interactive-Signage
```

You should now see a folder called `Sci-Tech-Discovery-Center-Interactive-Signage` on the Desktop.

Step 4 — Run the Installer

This step installs everything the kiosk needs to run. **It only needs to be done once.**

Open the project folder and double click the "setup" script file OR in the Terminal, type the following and press **Enter**:

```
./setup.sh
```

This takes about 5–10 minutes. Don't close the Terminal window while it runs.

When it finishes, you'll see the message: **Setup complete!**

If you see any error text**, the message should tell you in plain language what needs to be fixed.

Important:** Do not type `sudo` before `./setup.sh`. The installer handles that internally.

Step 5 — Connect the Physical Buttons and Scanner

Refer to [Sci-Tech Wiring Guide.pdf](#) for instructions on proper wiring connections to set up the system.

Starting and Stopping the Kiosk

If you've set up auto-start (see next section), you don't need to do any of this. Just plug the Pi in.

To start the kiosk manually

1. Open the Terminal.
2. Type the following and press **Enter**:

```
cd ~/Desktop/Sci-Tech-Discovery-Center-Interactive-Signage
```

3. Type this and press **Enter**:

```
./start.sh
```

The kiosk display will open on screen. Leave the Terminal window running in the background — don't close it.

To stop the kiosk

Click on the Terminal window and press **Ctrl + C** (hold the Control key and press C). The kiosk will close.

Make the Kiosk Start Itself on Power-Up (Recommended)

Once everything is working, you can set the kiosk to start automatically every time the Pi is turned on. This is highly recommended.

1. Open the project folder (Should be installed on your desktop from the previous steps)
2. Looks for the autostart enable script file
3. Double click the file to run
4. Autotart is now enabled, and the kiosk window will launch automatically at startup.

To turn off auto-start

Run the script file [autostart-disable.sh](#) by double clicking the icon.

If auto-start stops working

If the kiosk doesn't start automatically after a power cycle, you can check what went wrong:

1. Open the Terminal.
2. Type this and press **Enter**:

```
cat ~/Desktop/Sci-Tech-Discovery-Center-Interactive-Signage/signage.log
```

3. The error log will appear. Take a photo or screenshot and send it to the development team.

Swapping or Adding Videos

The easiest way to manage videos is the [NFC Tag Manager app](#) — see the next section. It handles file copying, naming, and registration automatically. The instructions below are for manually replacing video files outside of the app.

Note: For optimal performance, use a video with H.265 encoding at or below a resolution of 1920x1080 and a bit rate of 7 Mbps.

Where the videos live

Videos are stored inside the project folder. To find them using the file manager:

```
Desktop → Sci-Tech-Discovery-Center-Interactive-Signage → frontend → public → artifacts
```

Inside, you'll see one folder per organ (e.g., `heart`, `brain`, `lungs`). Each contains three video files:

Filename	Language
en.mp4	English
es.mp4	Spanish
te.mp4	Telugu

To replace a video manually

1. Copy the new video file onto the Pi (via USB drive, or download it to the Pi's browser).
2. Rename the new file to match the language it replaces — either `en.mp4` , `es.mp4` , Or `te.mp4` .
3. Drag it into the correct organ folder. Confirm when it asks if you want to replace the existing file.
4. After swapping any video files you must rebuild the project for the correct assets to be pulled. To do this, locate the setup script in the main project folder and double click it to run.

NFC Tag Manager — Full Guide

The **NFC Tag Manager** is a desktop app included with the kiosk software. It is the **recommended way** to add new artifacts, swap videos for existing artifacts, rename exhibits, or remove old ones — without ever editing files by hand.

What the app can do

- **Add a new artifact** — scan a fresh tag and pair it with an exhibit name plus videos for each language.
- **Edit an existing artifact** — change the exhibit name, swap any video file, or fill in a missing language.
- **Delete an artifact** — remove a tag and all its videos in one click.
- **See everything at a glance** — the main screen shows a scrollable list of every tag currently registered, with the exhibit name, UID, and which languages have videos.
- **Auto-copy video files** — when you select a video using the Browse button, the app copies it into the right folder with the right filename automatically. You don't need to know where files go.

Launching the app

Open the NFC manager folder, you should see a script file labeled "`launch.sh`", double click this to launch the app.

Or, alternatively

1. Open the Terminal.
2. Type this and press **Enter**:

```
cd ~/Desktop/Sci-Tech-Discovery-Center-Interactive-Signage/nfc_manager
```

3. Type this and press **Enter**:

```
./launch.sh
```

The app window will appear within a few seconds. The first time you launch it, it will install some additional pieces it needs — that's normal and only happens once.

If `./launch.sh` doesn't work for any reason, you can also run `python3 main.py` instead.

What you'll see

The window has two parts:

Left sidebar (purple)

- A logo and a status indicator showing whether the scanner is connected.
- *(On non-Pi computers only:)* A "Dev: Simulate Scan" panel at the bottom — a text box plus a Simulate button. Useful for practicing without real hardware.

Main area: "Manage Exhibits"

- A scrollable list of every artifact already configured. Each row shows:
 - **Exhibit name** (e.g., heart, brain, lungs) in large text
 - **UID** (the long number stored on the NFC tag)
 - **Videos** — which languages have a video file (e.g., "en, es, te")
 - **Edit** button — opens the editor for that artifact
 - **Delete** button (red) — removes the artifact and its videos

If no artifacts are configured yet, the main area will say "No exhibits configured. Scan a tag to add one."

Adding a new artifact

1. Hold a fresh, unregistered NFC tag near the scanner.
2. A pop-up window titled “**New Tag Detected**” will appear. It shows:
 - The **UID** of the tag (read-only, just for reference).
 - **Artifact Name** — type a short name like `heart` , `brain` , or `lungs` . Lowercase, no spaces. Spaces and capitals are converted automatically when you save (e.g., “Left Lung” becomes “left_lung”).
 - **Videos by Language** — three rows (English, Spanish, Telugu), each with a **Browse** button.
3. For each language, click **Browse**. A native file picker opens — navigate to the video file and click Open. The selected filename appears in the row.

Only `.mp4` video files can be selected.

4. Click **Save Exhibit**.

The app automatically:

- Copies each selected video into the right folder with the right name (`artifacts/heart/en.mp4` , etc.).
- Adds the new tag to the kiosk’s database.
- Refreshes the main list to show the new entry.

English is required** to save. Spanish and Telugu are optional — you can always add them later by editing the artifact.

Editing an existing artifact

You can edit an artifact two ways:

From the main list: click the **Edit** button on its row. **By scanning the tag:** hold the already-registered tag near the scanner. The same modal opens, pre-filled with the existing data.

In the editor:

- **Change the name** — type a new name in the Exhibit Name field. When you save, the app automatically renames the artifact’s folder and updates all the file paths.
- **Replace a video** — click Browse on the language row whose video you want to swap, pick the new file, and save. The new file is copied in, replacing the old one.
- **Add a missing language** — same as above; click Browse on the empty language row.
- **Leave a video unchanged** — just don’t click its Browse button. Existing videos stay as they are.

Click **Save Exhibit** to apply changes.

Deleting an artifact

1. Find the artifact in the main list.
2. Click its red **Delete** button.

The tag and all its associated video files are removed immediately. (*Currently there's no confirmation step — be careful.*)

After making changes

After adding, editing, or deleting artifacts in the manager, **restart the kiosk** to see the changes take effect:

```
cd ~/Desktop/Sci-Tech-Discovery-Center-Interactive-Signage
./stop.sh
./start.sh
```

(If auto-start is on, you can just unplug and re-plug the Pi.)

For Developers

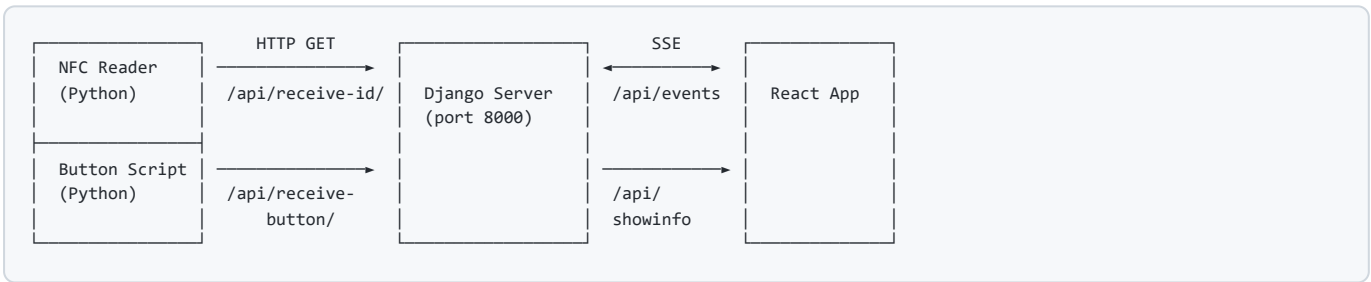
Project Structure

```
Sci-Tech-Discovery-Center-Interactive-Signage/
├── setup.sh                ← Top-level install script (Pi)
├── start.sh                ← Start all services + launch kiosk browser
├── stop.sh                 ← Kill all services
├── autostart-enable.sh     ← Configure desktop autostart (labwc or LXDE)
├── autostart-disable.sh   ← Remove autostart config
├── collection.json         ← Postman collection for API testing
├── frontend/              ← React + Vite UI
│   ├── src/
│   │   ├── App.jsx        ← Top-level state machine
│   │   ├── components/
│   │   │   ├── ScanPage.jsx ← Idle screen
│   │   │   ├── languageSelect.jsx ← Language picker
│   │   │   ├── videoScreen.jsx ← Video playback
│   │   │   └── Button.jsx    ← Shared button component
│   │   └── assets/testdata.json ← (legacy / dev fallback)
│   ├── public/artifacts/  ← Production video files (organ/lang structure)
│   └── dist/              ← Built output (gitignored)
├── interactive-signage-backend/ ← Django server
│   ├── polls/
│   │   ├── views.py       ← All API endpoints + SSE logic
│   │   ├── getpath.py     ← NFC ID → video path resolution
│   │   ├── testdata.json  ← Tag mapping (read by kiosk runtime)
│   │   └── exhibits.json  ← Tag mapping (written by NFC Manager)
│   └── mysite/settings.py
├── Hardware_Layer/        ← Pi hardware scripts (kiosk runtime)
│   ├── UIDRead_Updated.py ← PN532 NFC reader via SPI
│   ├── ButtonPress_Updated.py ← GPIO button listener
│   └── pn532/              ← PN532 driver library
├── nfc_manager/           ← Standalone tag-management desktop app
│   ├── main.py            ← Entry point; selects real vs mock reader
│   ├── launch.sh          ← Smart launcher (auto-runs setup if needed)
│   ├── setup.sh           ← Bootstraps venv + installs deps
│   ├── config.json        ← Path to signage project + language list
│   ├── requirements.txt    ← customtkinter, Pillow, darkdetect, spidev, rpi-lgpio
│   ├── tags.json          ← Legacy/unused (kept for backwards compat)
│   ├── data/manager.py    ← DataManager: reads/writes exhibits.json
│   ├── gui/
│   │   ├── app.py         ← Main App window + sidebar
│   │   ├── views.py       ← ManageTagsView (scrollable exhibit list)
│   │   ├── modals.py      ← TagModal (add/edit dialog)
│   │   └── theme.json      ← customtkinter brand theme
│   ├── nfc_reader/
│   │   ├── base.py        ← BaseNFCReader (threaded)
│   │   ├── pn532_reader.py ← Hardware reader via PN532 SPI
│   │   └── mock_reader.py  ← Dev simulate-scan reader
│   └── pn532/              ← PN532 driver (duplicate of Hardware_Layer's)
└── artifacts/             ← Design assets / per-organ doc folders
```

Tech Stack

Component	Technology
Display	React 19 + Vite + Tailwind CSS
Server	Django 5 (Python) with Server-Sent Events
Kiosk hardware layer	Python scripts using PN532 over SPI + RPi.GPIO
NFC Manager GUI	Python + customtkinter, Pillow, darkdetect
Communication	HTTP GET (hardware → server) + SSE (server → browser)
Storage	JSON config + local <code>.mp4</code> files

Architecture & API



The hardware scripts send scan/button events to Django via HTTP GET. The React frontend holds a persistent SSE connection and drives the UI state machine: `idle` → `scanned` → `language_select` → `playing` → `idle`.

Endpoints

Endpoint	Method	Description
<code>/api/receive-id/?id=<id></code>	GET	Register NFC scan; starts 15s language timeout
<code>/api/receive-button/?button=a b c</code>	GET	Button press (<code>a</code> =EN, <code>b</code> =ES, <code>c</code> =TE)
<code>/api/showinfo/</code>	GET	Returns video path for current scan + language
<code>/api/resetinfo/</code>	GET	Resets server state for next visitor
<code>/api/events/</code>	GET	SSE stream — events: <code>scanned_id</code> , <code>button_press</code> , <code>button_press_timeout</code>

A **Postman collection** (`collection.json`) is included at the repo root. Import it into Postman and point it at `http://localhost:8000` for manual API testing.

NFC Tag ID Format

IDs are decimal strings, derived in `pn532_reader.py` via:

```
str(int.from_bytes(uid_bytes, byteorder='big'))
```

Don't convert hex UUIDs from third-party tools byte order will differ.

The hardware reader debounces repeat scans of the same tag within 3 seconds.

NFC Manager Internals

- `nfc_manager/main.py` selects between `PN532NFCReader` (Pi) and `MockNFCReader` (Mac/Windows or `USE MOCK_READER=1`) at startup.
- `BaseNFCReader` runs polling in a daemon thread; tag callbacks are dispatched via `App.after(0, ...)` to keep Tk happy.
- `DataManager` is the single source of truth for tag CRUD. It reads `config.json` to locate the signage project, then reads/writes `exhibits.json` and copies videos into `artifacts/<name>/<lang>.mp4`.
- `TagModal` validates that English is selected before allowing save. Names are normalized via `.strip().lower().replace(" ", "_")`.

Running in Development (not on a Pi)

The kiosk's hardware scripts skip themselves automatically when `/dev/spidev0.0` is absent. On-screen language buttons work without physical hardware.

```
# Terminal 1 - backend
cd interactive-signage-backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python manage.py runserver
```

```
# Terminal 2 - frontend (hot reload)
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173` (Vite dev server with HMR) or `http://localhost:8000` (Django serving the last built frontend).

Simulate events:

```
curl "http://localhost:8000/api/receive-id?id=1212866967841409"
curl "http://localhost:8000/api/receive-button?button=a" # English
curl "http://localhost:8000/api/receive-button?button=b" # Spanish
curl "http://localhost:8000/api/receive-button?button=c" # Telugu
```

For the NFC Manager in dev mode:

```
cd nfc_manager
./launch.sh
# Or force mock reader explicitly:
USE MOCK_READER=1 python3 main.py
```

The “Dev: Simulate Scan” panel will appear in the sidebar — type any UID and click Simulate.

Node.js version note

`apt install nodejs` on some Pi OS versions installs Node 12, which will break the Vite build. Check with `node --version` — Node 18+ is required. If needed:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```
